UNIVERSITI MALAYSIA TERENGGANU

FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONORS

SEMESTER II 2025/2026

CSA3023
WEB-BASED APPLICATION DEVELOPMENT

LAB 5:
JSP: JAVABEANS AND STANDARD TAG LIBRARY (JSTL)

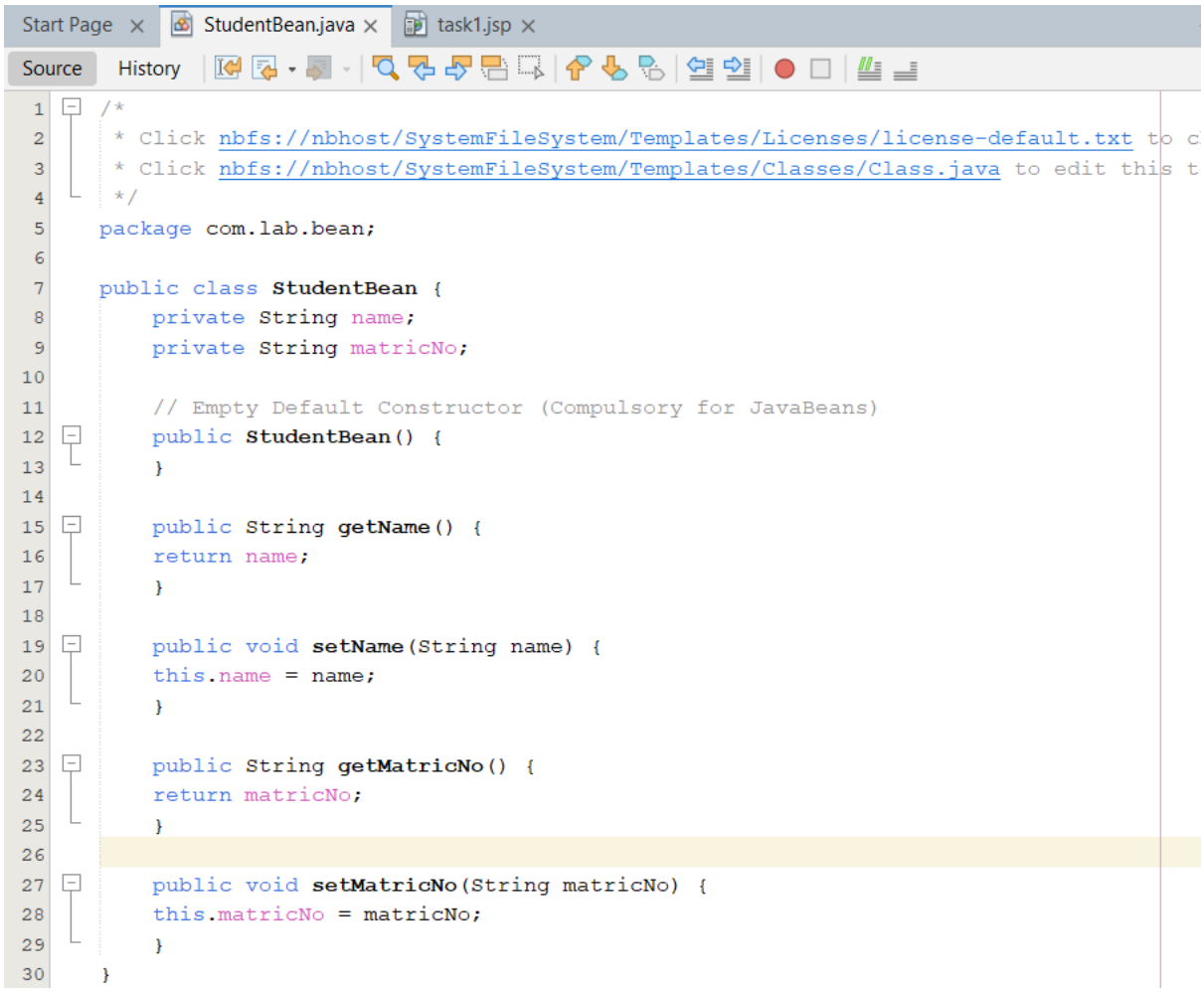
Prepared by:
NURSYAHEERA BINTI MOHD HISHAM

Prepared for:
SIR. MOHAMAD ARIZAL SHAMSIL MAT RIFFIN

Date:
12 MAY 2026

Task 1: Using Scriptlet to Access a Simple JavaBeans

Objective: To understand how to create a basic JavaBean and instantiate it inside a JSP page using standard Java scriptlets.

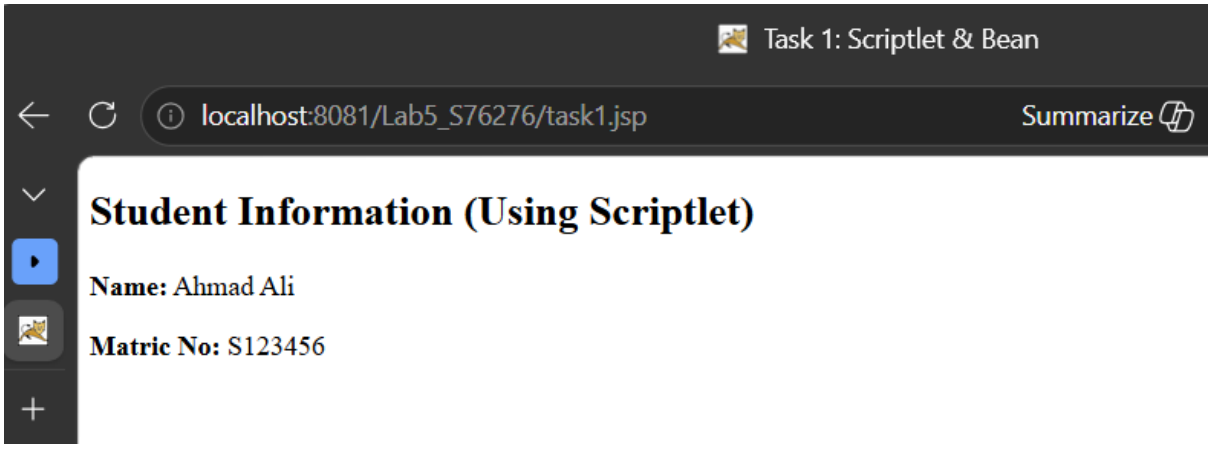


Start Page | StudentBean.java | task1.jsp

Source | History

```jsp
<%--
    Document   : task1
    Created on : 12 May 2026, 4:57:06 pm
    Author     : nursyaheera binti mohd hisham
--%>

<%@page import="com.lab.bean.StudentBean"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <title>Task 1: Scriptlet & Bean</title>
    </head>
    <body>
    <h2>Student Information (Using Scriptlet)</h2>

    <%
        // Instantiating the bean
        StudentBean student = new StudentBean();
        student.setName("Ahmad Ali");
        student.setMatricNo("S123456");
    %>

    <p><strong>Name:</strong> <%= student.getName() %></p>
    <p><strong>Matric No:</strong> <%= student.getMatricNo() %></p>
    </body>
</html>
```

Output:



Reflections

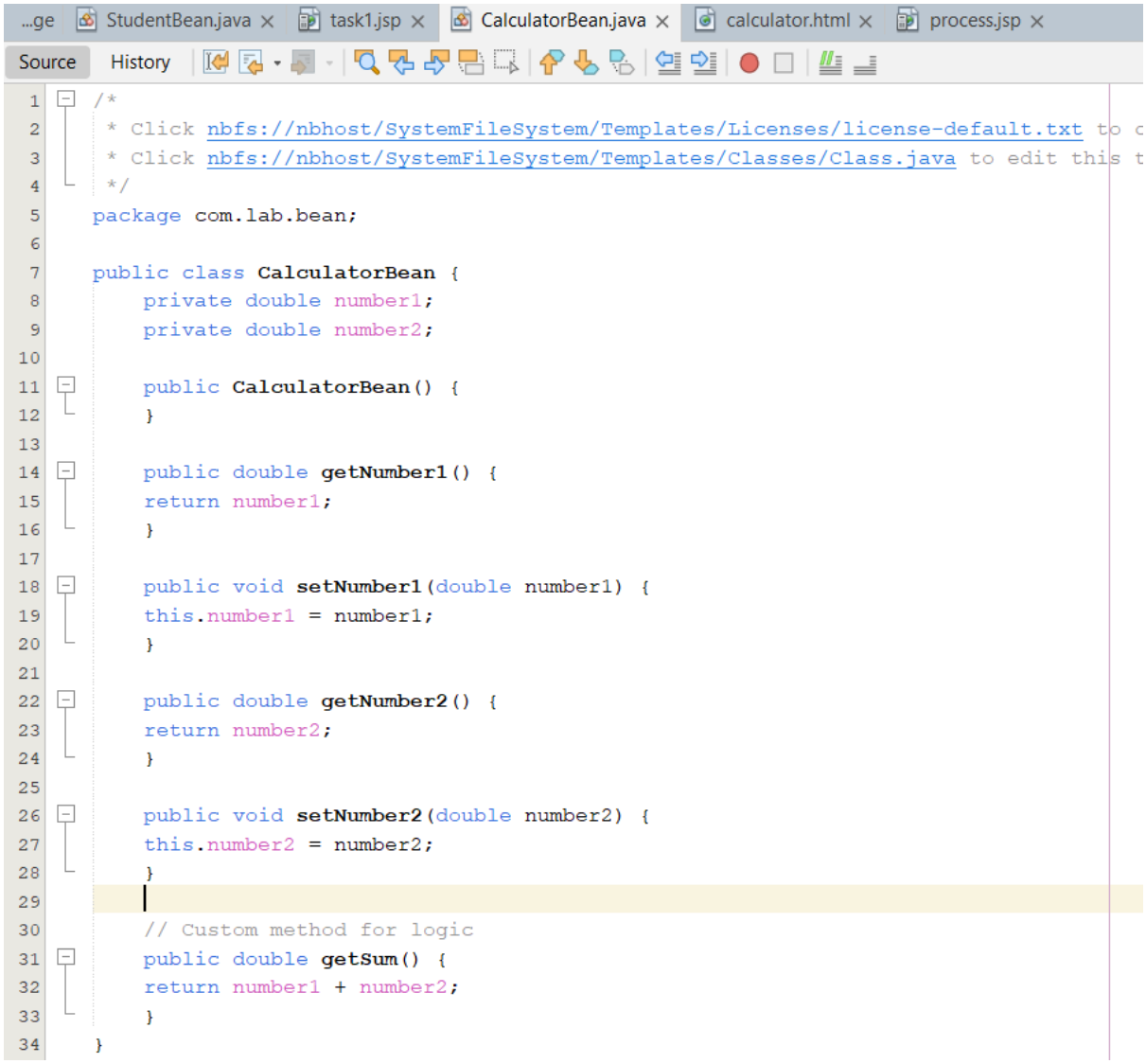
1. What is the main disadvantage of mixing Java scriptlets directly inside HTML codes?
   Answer:
   The main disadvantage is that it makes the JSP page difficult to read, maintain, and debug because Java code and HTML code are mixed together. It also reduces code reusability and violates the separation between business logic and presentation.

2. Why is it compulsory for a JavaBean class to have an empty default constructor?
   Answer:

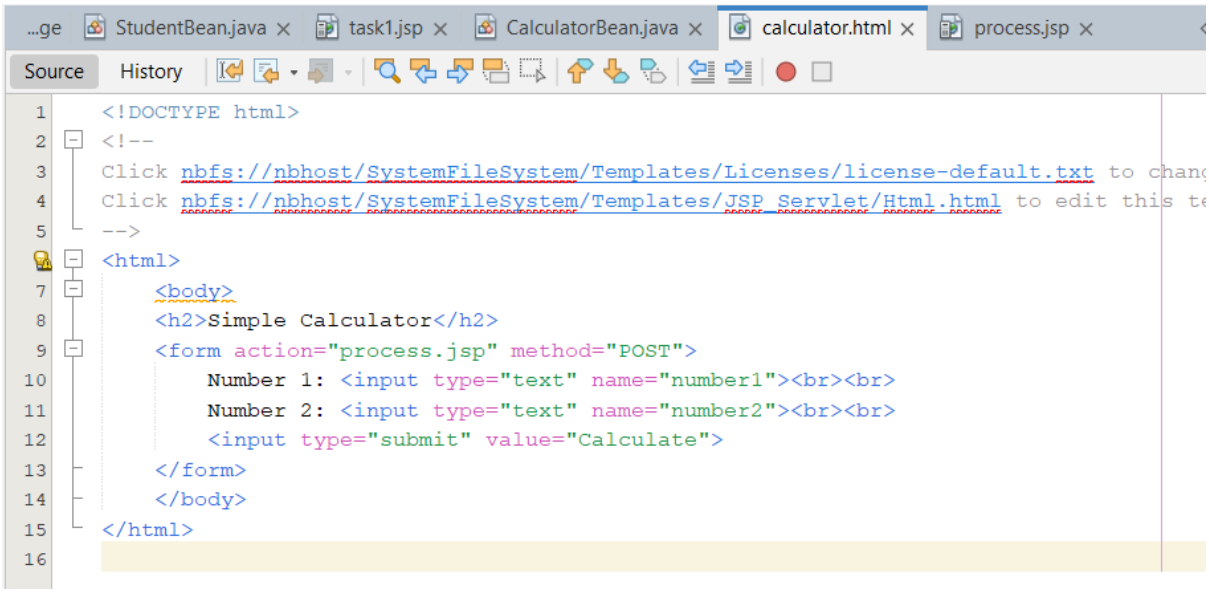
An empty default constructor is compulsory because JSP and JavaBeans use it to automatically create an object instance. Without the default constructor, the bean cannot be instantiated properly by JSP action tags such as <jsp:useBean>.

Task 2: Problem Solving using JavaBeans

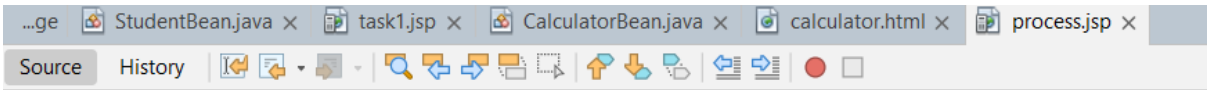
Objective: To utilize JSP standard action tags (,) to process HTML form data automatically without using scriptlets.




```java
/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to c
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this t
 */
package com.lab.bean;

public class CalculatorBean {
    private double number1;
    private double number2;

    public CalculatorBean() {
    }

    public double getNumber1() {
    return number1;
    }

    public void setNumber1(double number1) {
    this.number1 = number1;
    }

    public double getNumber2() {
    return number2;
    }

    public void setNumber2(double number2) {
    this.number2 = number2;
    }

    // Custom method for logic
    public double getSum() {
    return number1 + number2;
    }
}
```

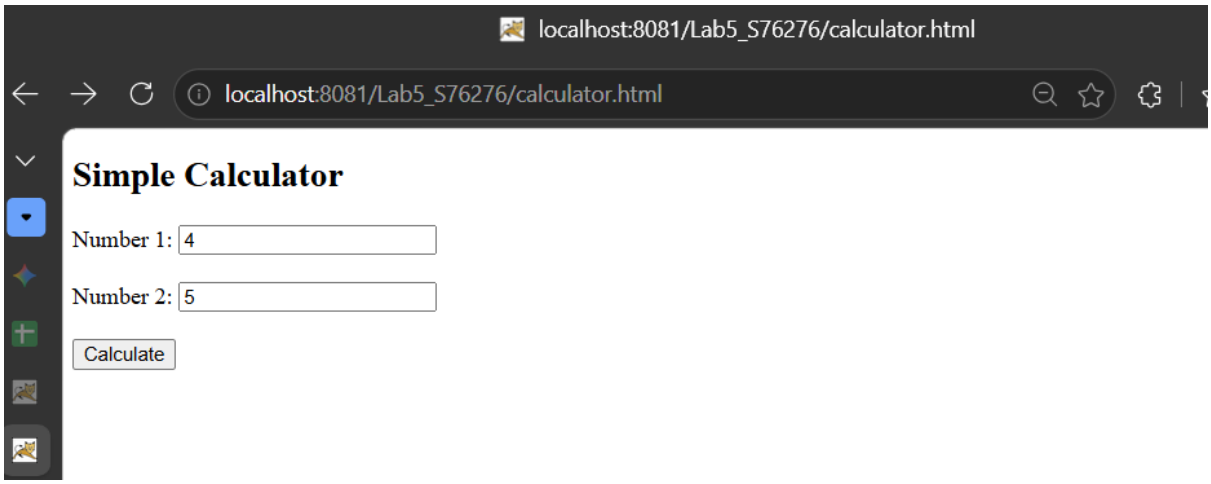


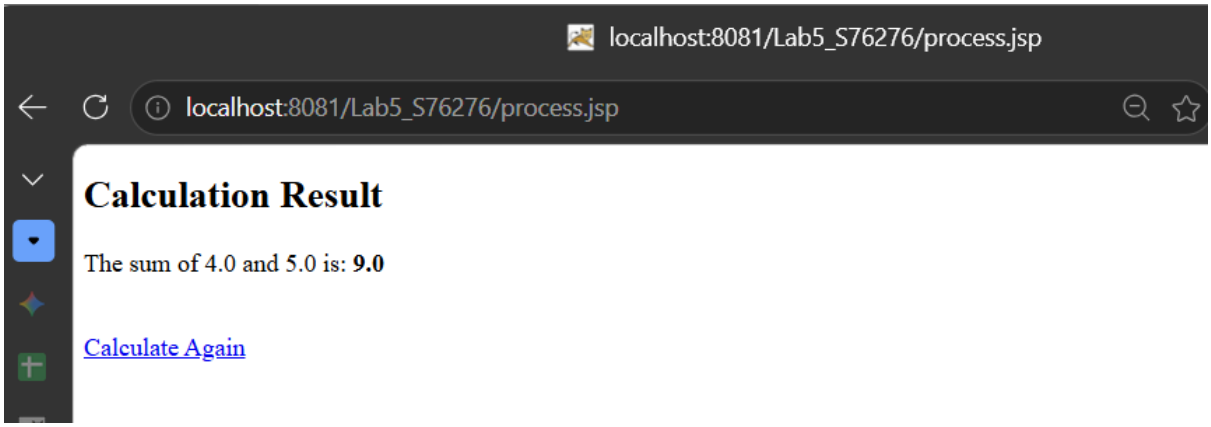

```
<!DOCTYPE html>
<!--
Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to chang
Click nbfs://nbhost/SystemFileSystem/Templates/JSP_Servlet/Html.html to edit this te
-->
<html>
    <body>
    <h2>Simple Calculator</h2>
    <form action="process.jsp" method="POST">
        Number 1: <input type="text" name="number1"><br><br>
        Number 2: <input type="text" name="number2"><br><br>
        <input type="submit" value="Calculate">
    </form>
    </body>
</html>
```




```
<%--
    Document   : process
    Created on : 12 May 2026, 5:09:05 pm
    Author     : nursyaheera binti mohd hisham
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <body>
        <h2>Calculation Result</h2>

        <jsp:useBean id="calc" class="com.lab.bean.CalculatorBean" />

        <jsp:setProperty name="calc" property="*" />

        <p>The sum of
            <jsp:getProperty name="calc" property="number1" /> and
            <jsp:getProperty name="calc" property="number2" /> is:
            <strong><jsp:getProperty name="calc" property="sum" /></strong>
        </p>

        <br><a href="calculator.html">Calculate Again</a>
    </body>
</html>
```

Output:





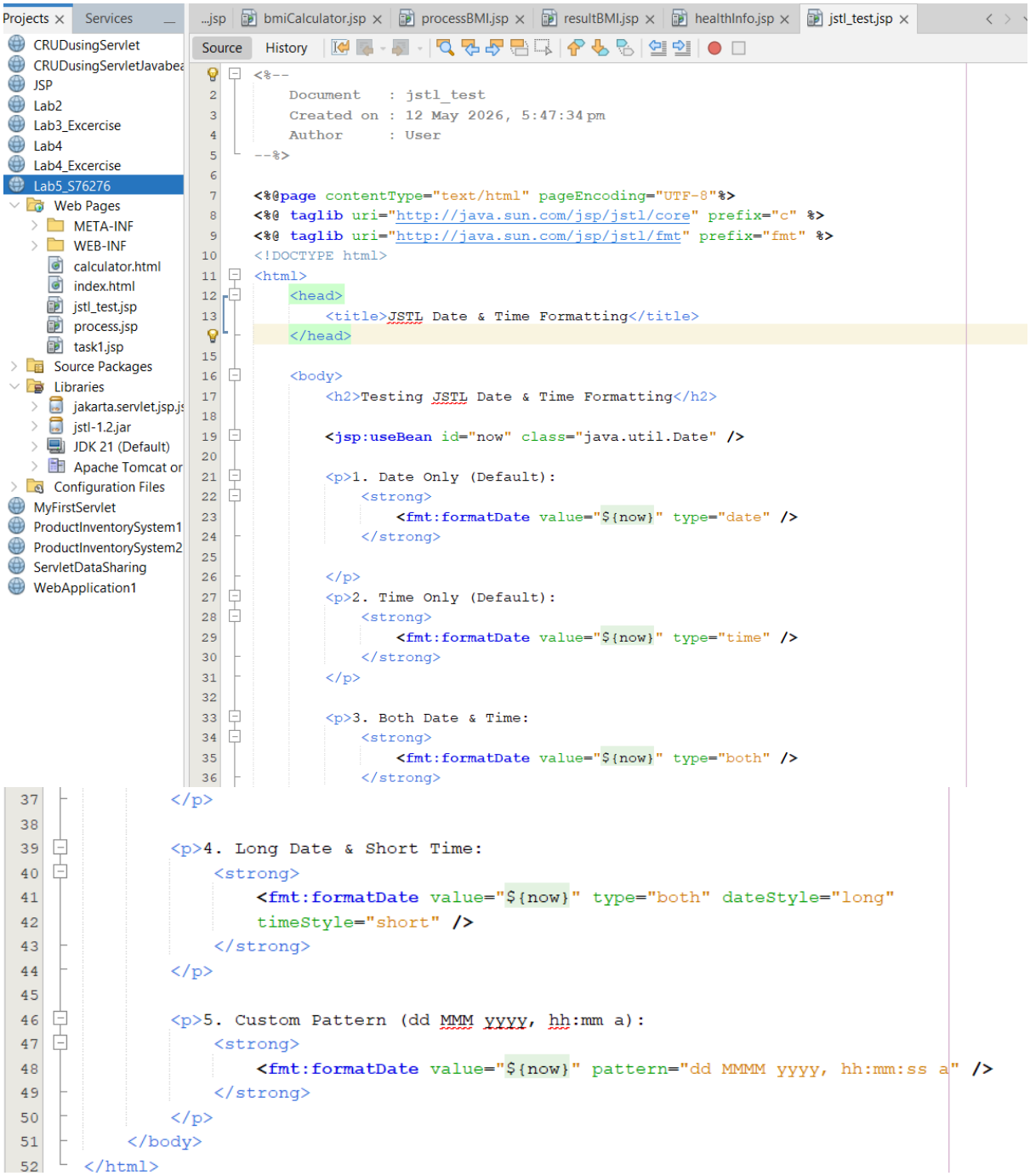
Reflections

1. Explain how the property="*" attribute works in the tag.
   Answer:
   The property="*" attribute automatically matches the HTML form input names with the JavaBean property names. If the form field name and bean variable name are the same, the values are automatically assigned to the bean without manually calling setter methods.

2. How do JSP action tags improve code readability compared to the scriptlets used in Task 1?
   Answer:
   JSP action tags improve readability because they reduce the amount of Java code written inside the JSP page. The code becomes cleaner, easier to understand, and easier to maintain since the logic is handled automatically using predefined tags.

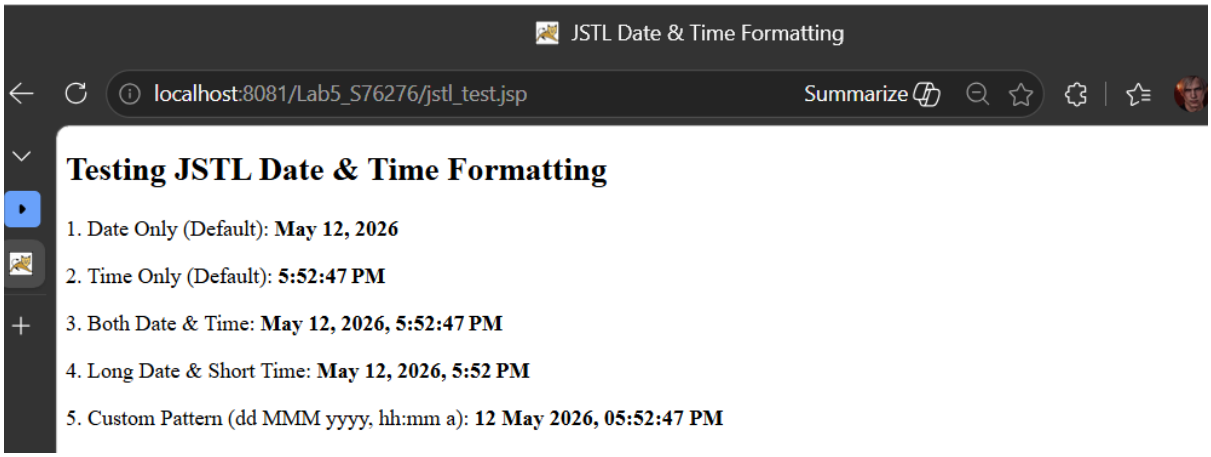
Task 3: Installing JSTL Taglibs

Objective: To download, install, configure, and test the JavaServer Pages Standard Tag Library (JSTL) in a web project.




```
            </p>

            <p>4. Long Date & Short Time:
                <strong>
                    <fmt:formatDate value="${now}" type="both" dateStyle="long"
                    timeStyle="short" />
                </strong>
            </p>

            <p>5. Custom Pattern (dd MMM yyyy, hh:mm a):
                <strong>
                    <fmt:formatDate value="${now}" pattern="dd MMMM yyyy, hh:mm:ss a" />
                </strong>
            </p>
        </body>
    </html>
```

Output:



Reflections

1. What is the purpose of the uri and prefix attributes in the taglib directive?
   Answer:
   The uri attribute identifies the JSTL library being used, while the prefix attribute defines the shortcut name used to access JSTL tags inside the JSP page.

2. Why do we need separate taglib directives for core and fmt?
   Answer:
   Separate taglib directives are needed because each JSTL library provides different functions. The Core library handles logic and iteration tags, while the Format (fmt) library handles formatting for numbers, dates, currencies, and localization.

Task 4: Using Java Standard Tag Library (JSTL) Core Tags

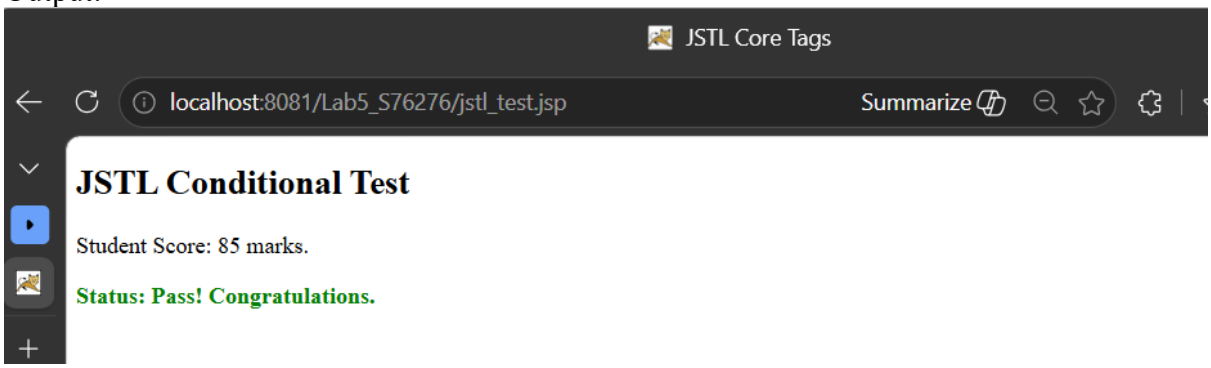
Objective: To perform variable declarations, value output, and conditional checks using JSTL Core Tags

...jsp | bmiCalculator.jsp | processBMI.jsp | resultBMI.jsp | healthInfo.jsp | jstl_test.jsp

Source | History

```jsp
<%--
    Document   : jstl_test
    Created on : 12 May 2026, 5:47:34 pm
    Author     : nursyaheera
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
    <title>JSTL Core Tags</title>
</head>
<body>
    <h2>JSTL Conditional Test</h2>
    <c:set var="score" value="85" />

    <p>Student Score: <c:out value="${score}" /> marks.</p>

    <c:if test="${score >= 50}">
        <p style="color: green;"><strong>Status: Pass! Congratulations.</strong>
        </p>
    </c:if>

    <c:if test="${score < 50}">
        <p style="color: red;"><strong>Status: Fail. Please try again.</strong>
        </p>
    </c:if>
</body>
</html>
```

Output:



Reflections

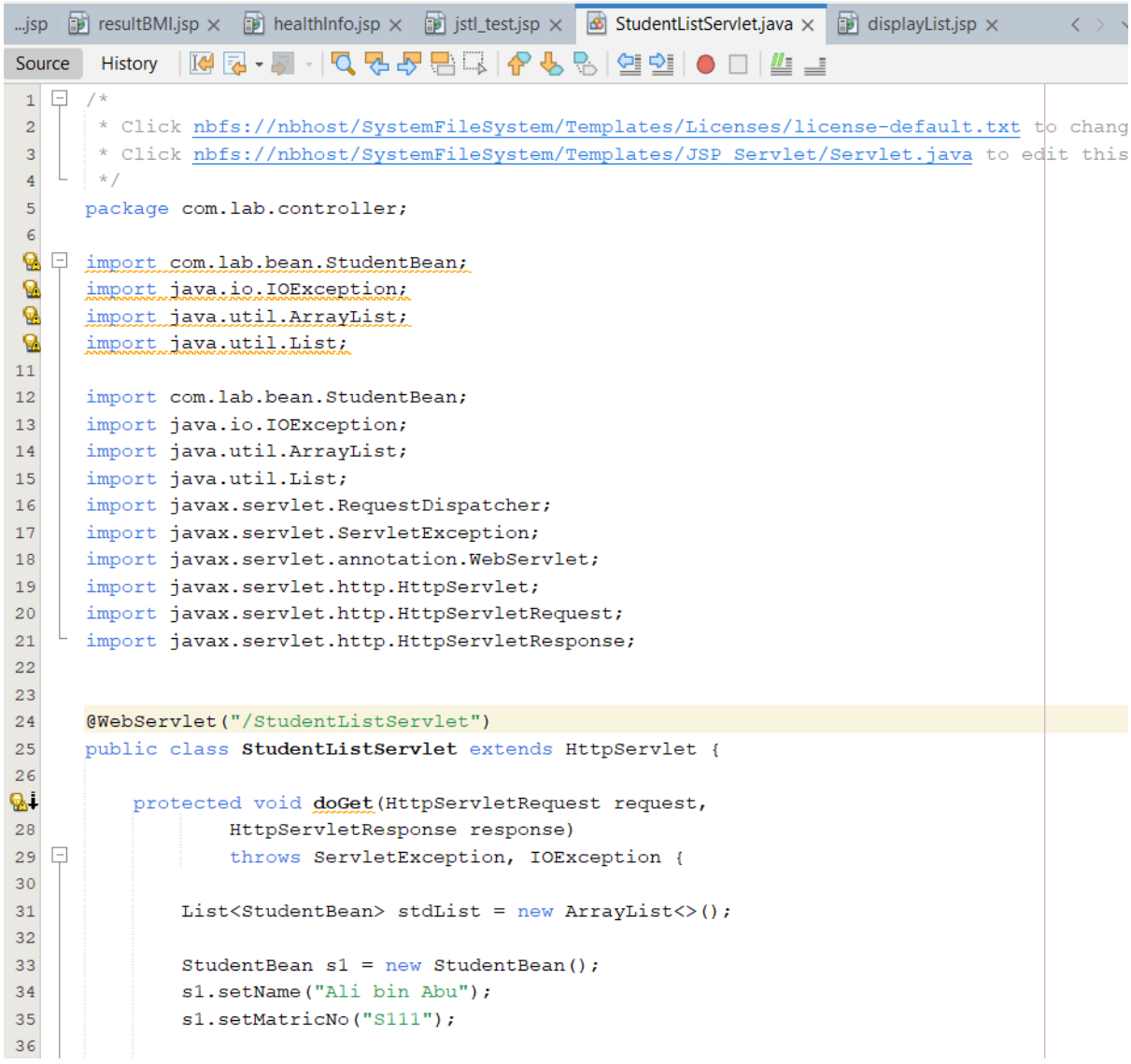
1. How does the tag help in preventing Cross-Site Scripting (XSS) attacks?
   Answer:

The <c:out> tag automatically escapes special HTML characters before displaying output. This prevents malicious scripts from being executed in the browser, helping to reduce XSS attacks.

2. The tag does not have an ”else” statement. Which JSTL tags should be used if you need an if-else logic structure?
   Answwr:
   The JSTL tags <c:choose>, <c:when>, and <c:otherwise> should be used to create an if-else logic structure.

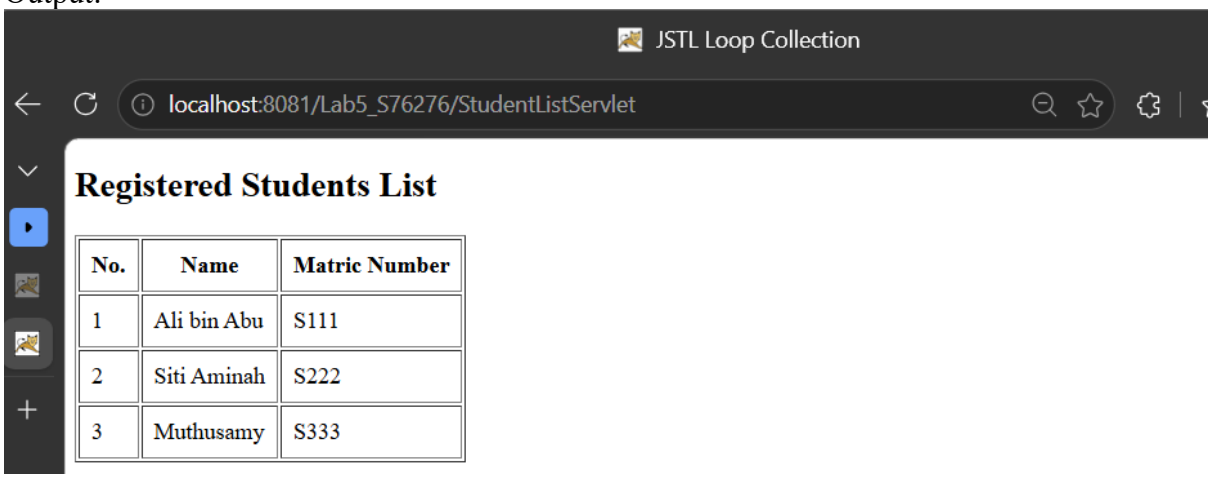
Task 5: Using JSP Standard Tag Library (JSTL) for Collections

Objective: To combine JavaBeans and JSTL by iterating through a collection of objects using the loop.



```
37              StudentBean s2 = new StudentBean();
38              s2.setName("Siti Aminah");
39              s2.setMatricNo("S222");
40
41              StudentBean s3 = new StudentBean();
42              s3.setName("Muthusamy");
43              s3.setMatricNo("S333");
44
45              stdList.add(s1);
46              stdList.add(s2);
47              stdList.add(s3);
48
49              request.setAttribute("listData", stdList);
50
51              RequestDispatcher rd =
52                      request.getRequestDispatcher("displayList.jsp");
53
54              rd.forward(request, response);
55          }
56      }
```

Output:



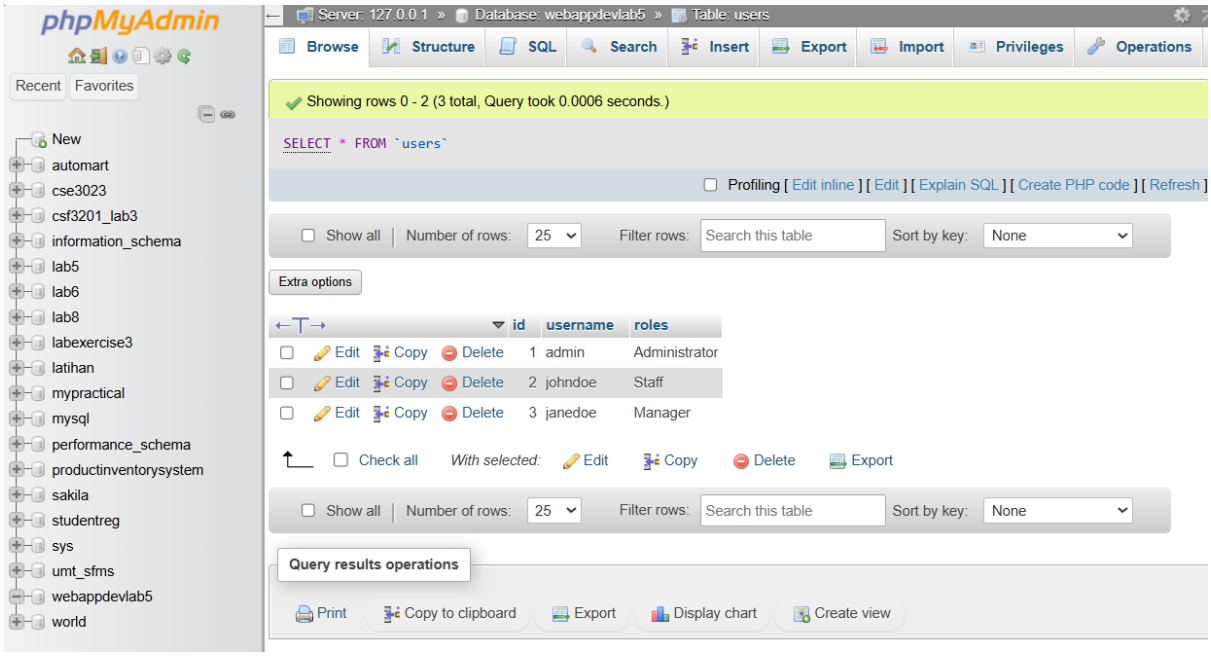
Reflections

1. Describe how the items and var attributes work together in the tag.
   Answer:
   The items attribute represents the collection or list to be looped through, while the var attribute stores each individual object during every iteration of the loop.

2. Why is it important to run the Servlet instead of the JSP file directly in this task?
   Answer:
   It is important because the Servlet prepares and sends the data (listData) to the JSP page. If the JSP file is run directly, the required data will not exist, causing empty output or errors.

Task 6: Database Access using JSTL SQL Tags

Objective: To connect to a MySQL database and execute SQL queries directly from a JSP page using the JSTL SQL Tag Library.

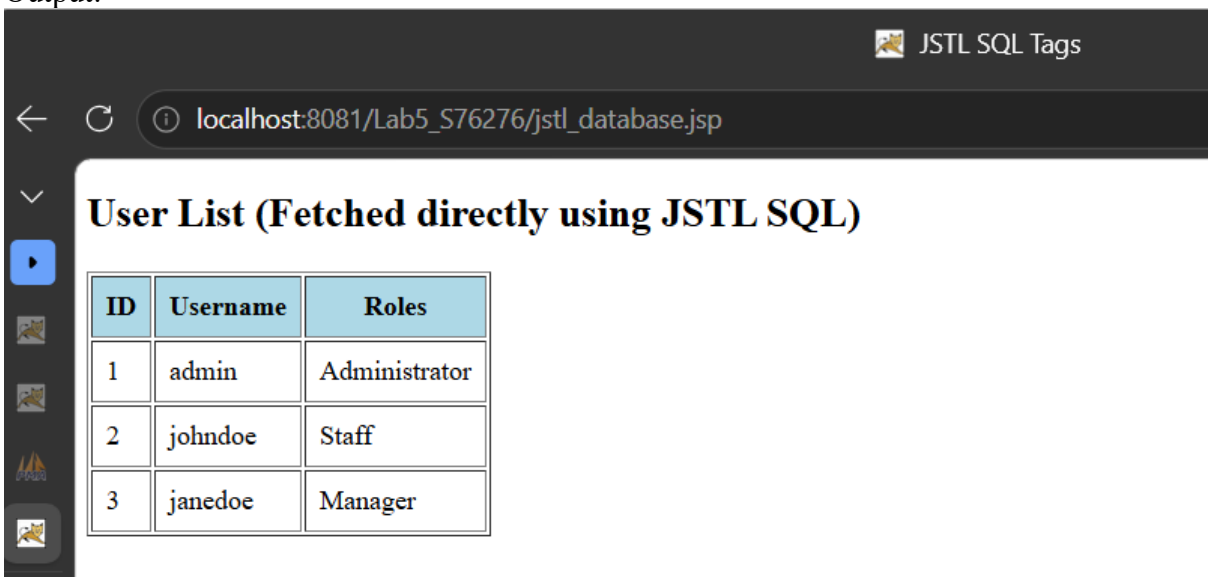


...jsp | healthInfo.jsp | jstl_test.jsp | StudentListServlet.java | displayList.jsp | jstl_database.jsp

Source | History

```jsp
<%--
    Document   : jstl_database
    Created on : 12 May 2026, 6:19:59 pm
    Author     : nursyaheera
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/sql" prefix="sql" %>
<!DOCTYPE html>
<html>
<head>
    <title>JSTL SQL Tags</title>
</head>
<body>
    <h2>User List (Fetched directly using JSTL SQL)</h2>
    <sql:setDataSource var="dbConnection" driver="com.mysql.cj.jdbc.Driver"
        url="jdbc:mysql://localhost:3306/webappdevlab5"
        user="root" password="admin"/>

    <sql:query dataSource="${dbConnection}" var="result">
        SELECT * FROM users;
    </sql:query>
    <table border="1" cellpadding="8">
        <thead>
            <tr style="background-color: lightblue;">
            <th>ID</th>
            <th>Username</th>
            <th>Roles</th>
            </tr>
        </thead>
    <tbody>
        <c:forEach var="row" items="${result.rows}">
            <tr>
                <td><c:out value="${row.id}" /></td>
                <td><c:out value="${row.username}" /></td>
                  <td><c:out value="${row.roles}" /></td>
                </tr>
         </c:forEach>
     </tbody>
     </table>
</body>
</html>
```

Output:



Reflections

a. The JSTL tag allows you to retrieve data without writing Servlets or DAO classes. Why is this considered a bad practice (violating the MVC pattern) for large-scale enterprise applications?

Answer:
This is considered bad practice because it mixes database logic directly inside the presentation layer (JSP). It violates the MVC architecture where database operations should belong to the Model layer. In large applications, this approach makes the code difficult to maintain, reuse, secure, and debug. Using Servlets and DAO classes provides better organization, scalability, and separation of responsibilities.

Exercise: Employee Payroll Display System

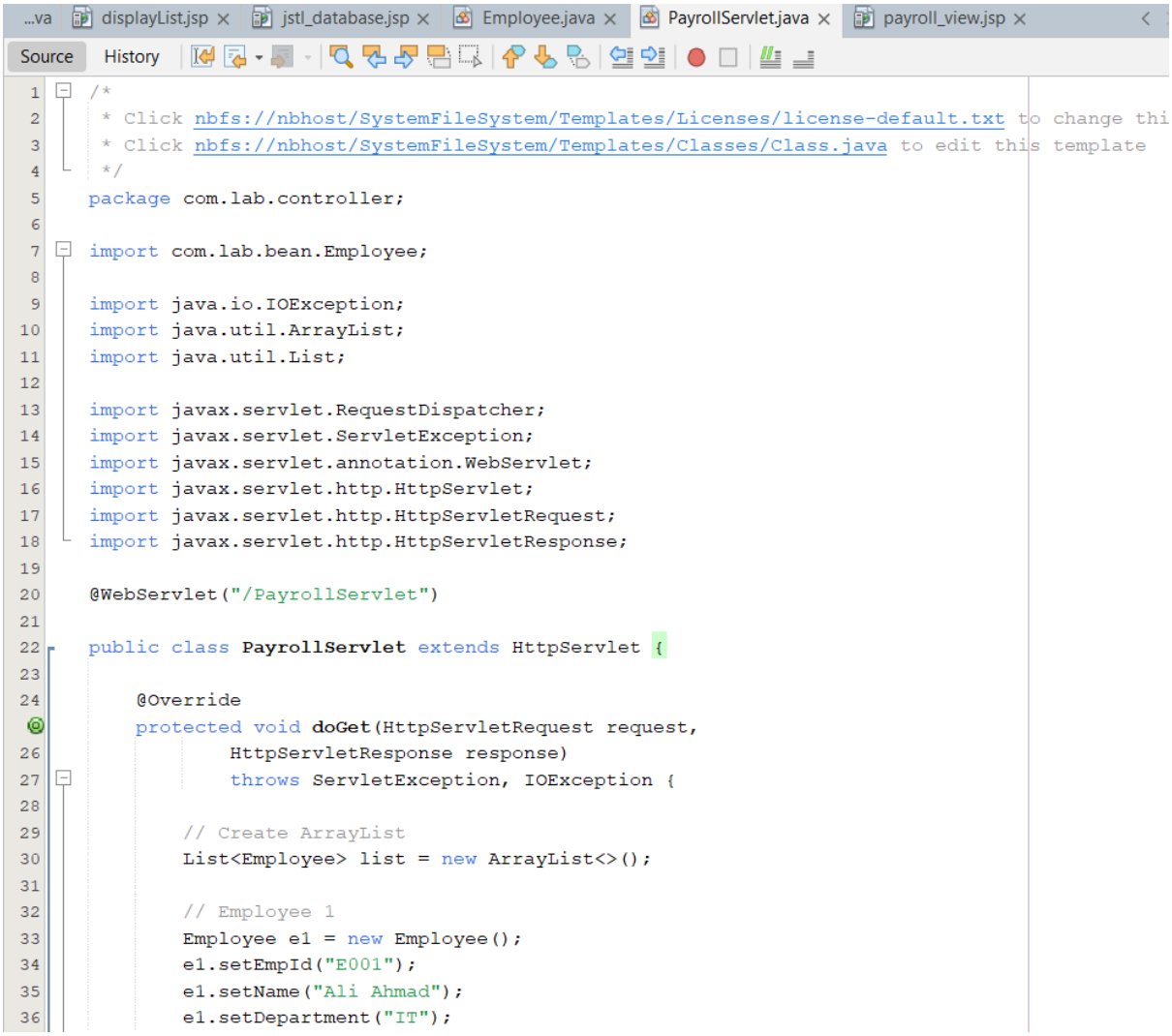
Based on the concepts you have learned in this lab (JavaBeans, Servlets as Controllers, and JSTL for the View), you are required to independently develop a simple Employee Payroll Display System.
Please complete this exercise by fulfilling the following requirements:

1. JavaBean (Model): Create a class named Employee in the com.lab.bean package with the following private attributes: empId (String), name (String), department (String), and basicSalary (double). Include the standard empty constructor and the getter/setter methods for all attributes.

2. Servlet (Controller): Create a PayrollServlet in the com.lab.controller package. Inside this Servlet's doGet() method:
   • Create a list (ArrayList) and populate it with at least 5 different Employee objects.
   • Share this list to the JSP using request.setAttribute("employeeList", list);.
   • Forward the request to a view file named payroll_view.jsp.

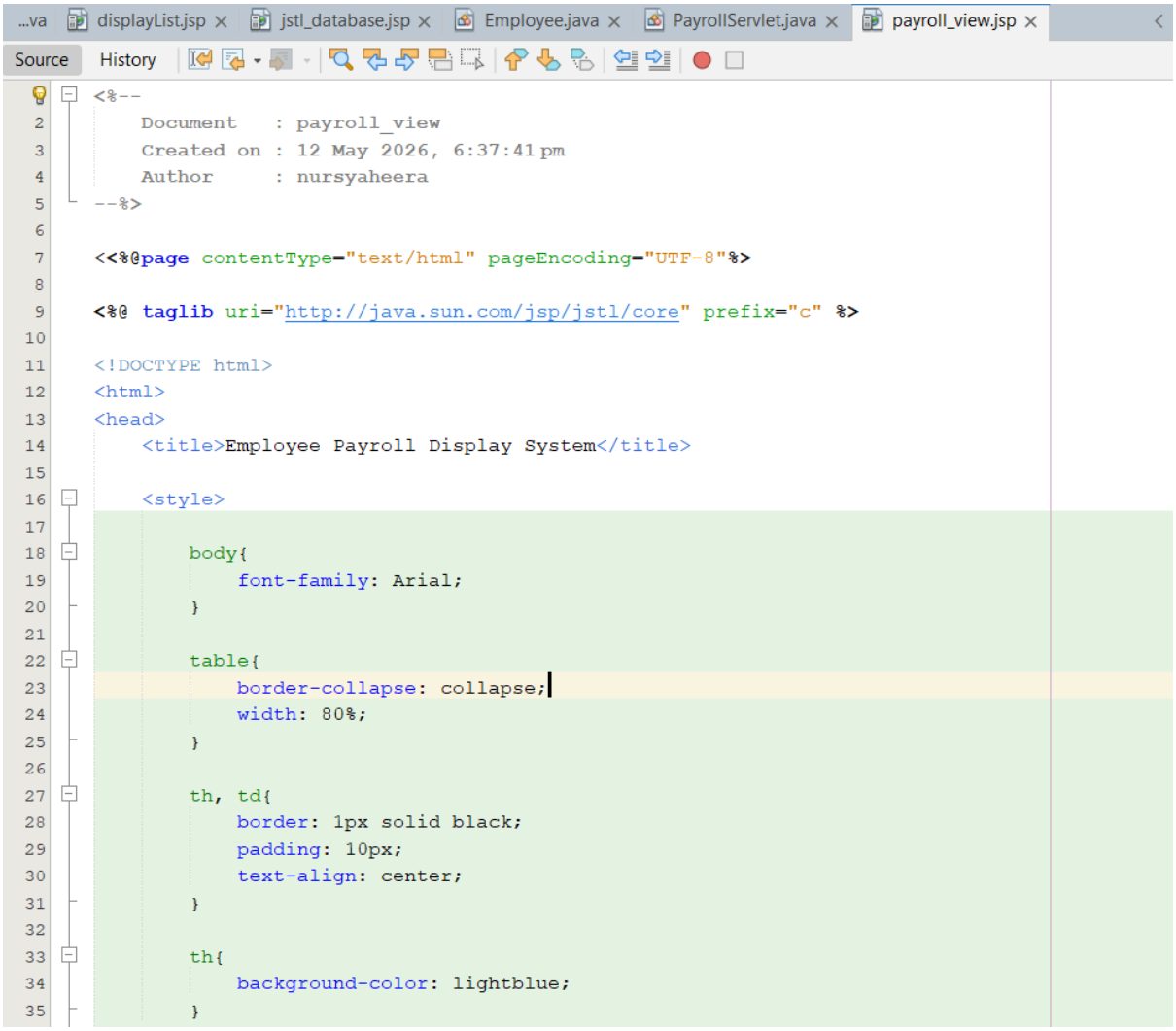
3. JSP & JSTL (View): Create payroll_view.jsp to display the data:
   • Import the JSTL core taglib directive at the very top of the file.
   • Use the tag to iterate through the employeeList and display their details in a well-formatted HTML table.
   • Logic Challenge: Use the , , and tags inside the table to add a new "Status" column. If the employee's basic salary is RM3000 or above, display the text "Senior"; otherwise, display "Junior".


...va
displayList.jsp
jstl_database.jsp
Employee.java
PayrollServlet.java
payroll_view.jsp
Source
History
/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change th
* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
*/
package com.lab.bean;
public class Employee {
private String empId;
private String name;
private String department;
private double basicSalary;
// Empty Constructor
public Employee() {
}
// Getter and Setter Methods
public String getEmpId() {
return empId;
}
public void setEmpId(String empId) {
this.empId = empId;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
public String getDepartment() {
return department;
}
public void setDepartment(String department) {
this.department = department;
}
public double getBasicSalary() {
return basicSalary;
}
public void setBasicSalary(double basicSalary) {
this.basicSalary = basicSalary;
}
}


...va
displayList.jsp
jstl_database.jsp
Employee.java
PayrollServlet.java
payroll_view.jsp
Source
History
/*
* Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to change thi
* Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this template
*/
package com.lab.controller;
import com.lab.bean.Employee;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import javax.servlet.RequestDispatcher;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
@WebServlet("/PayrollServlet")
public class PayrollServlet extends HttpServlet {
@Override
protected void doGet(HttpServletRequest request,
HttpServletResponse response)
throws ServletException, IOException {
// Create ArrayList
List<Employee> list = new ArrayList<>();
// Employee 1
Employee e1 = new Employee();
e1.setEmpId("E001");
e1.setName("Ali Ahmad");
e1.setDepartment("IT");

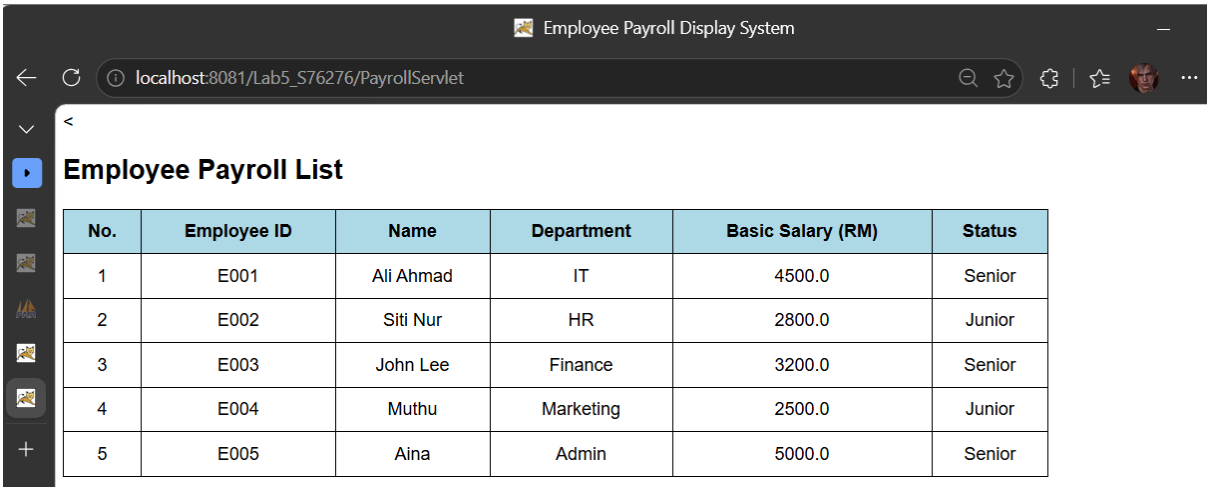
```
        e1.setBasicSalary(4500);

        // Employee 2
        Employee e2 = new Employee();
        e2.setEmpId("E002");
        e2.setName("Siti Nur");
        e2.setDepartment("HR");
        e2.setBasicSalary(2800);

        // Employee 3
        Employee e3 = new Employee();
        e3.setEmpId("E003");
        e3.setName("John Lee");
        e3.setDepartment("Finance");
        e3.setBasicSalary(3200);

        // Employee 4
        Employee e4 = new Employee();
        e4.setEmpId("E004");
        e4.setName("Muthu");
        e4.setDepartment("Marketing");
        e4.setBasicSalary(2500);

        // Employee 5
        Employee e5 = new Employee();
        e5.setEmpId("E005");
        e5.setName("Aina");
        e5.setDepartment("Admin");
        e5.setBasicSalary(5000);

        // Add into list
        list.add(e1);
        list.add(e2);
        list.add(e3);
        list.add(e4);
        list.add(e5);
```

```
        list.add(e3);
        list.add(e4);
        list.add(e5);

        // Share data with JSP
        request.setAttribute("employeeList", list);

        // Forward to JSP
        RequestDispatcher rd =
                request.getRequestDispatcher("payroll_view.jsp");

        rd.forward(request, response);
    }
}
```


...va
displayList.jsp
jstl_database.jsp
Employee.java
PayrollServlet.java
payroll_view.jsp
Source
History
<%--
Document : payroll_view
Created on : 12 May 2026, 6:37:41 pm
Author : nursyaheera
--%>
<<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<!DOCTYPE html>
<html>
<head>
<title>Employee Payroll Display System</title>
<style>
body{
font-family: Arial;
}
table{
border-collapse: collapse;
width: 80%;
}
th, td{
border: 1px solid black;
padding: 10px;
text-align: center;
}
th{
background-color: lightblue;
}

```
    </style>

</head>

<body>

    <h2>Employee Payroll List</h2>

    <table>

        <thead>

            <tr>
                <th>No.</th>
                <th>Employee ID</th>
                <th>Name</th>
                <th>Department</th>
                <th>Basic Salary (RM)</th>
                <th>Status</th>
            </tr>

        </thead>

        <tbody>

            <c:forEach var="emp"
                       items="${employeeList}"
                       varStatus="status">

                <tr>

                    <td>${status.count}</td>

                    <td>${emp.empId}</td>

                    <td>${emp.name}</td>

                    <td>${emp.department}</td>

                    <td>${emp.basicSalary}</td>

                    <td>

                        <c:choose>

                            <c:when test="${emp.basicSalary >= 3000}">
                                Senior
                            </c:when>

                            <c:otherwise>
                                Junior
                            </c:otherwise>

                        </c:choose>

                    </td>

                </tr>

            </c:forEach>

        </tbody>

    </table>

</body>
</html>
```

Output:


Employee Payroll Display System
localhost:8081/Lab5_S76276/PayrollServlet
<
Employee Payroll List
No. | Employee ID | Name | Department | Basic Salary (RM) | Status
1 | E001 | Ali Ahmad | IT | 4500.0 | Senior
2 | E002 | Siti Nur | HR | 2800.0 | Junior
3 | E003 | John Lee | Finance | 3200.0 | Senior
4 | E004 | Muthu | Marketing | 2500.0 | Junior
5 | E005 | Aina | Admin | 5000.0 | Senior